

Memory and Multithreading Optimization for Multi-Agent Pathfinding

Benjamin Shaffrey

Academy of Engineering and Technology

Introduction

- This project aims to produce an algorithm which can solve the multi-agent pathfinding (MAPF) problem as quickly as possible.
- Standard pathfinding seeks to find between nodes on a graph (see figure 1)

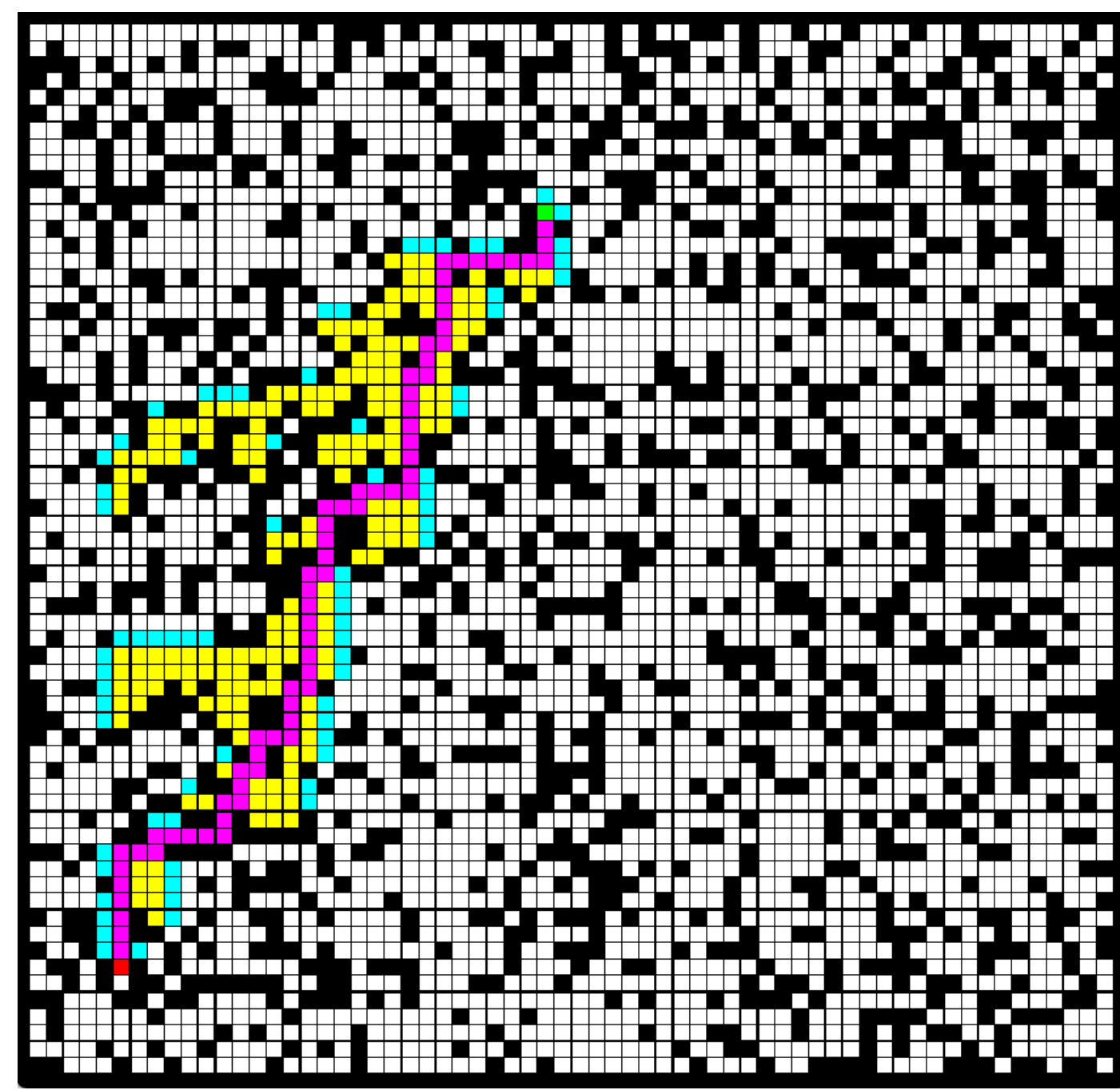


Figure 1. Visualization of standard pathfinding

- MAPF seeks to find sets of paths with no 'crashes': two agents occupying the same location at the same time step
- MAPF is often very computationally intense to solve compared to standard pathfinding, as the number of possible solutions is exponentially larger
- Speed of solution is increasingly important for practical use in fields like automation and video games
- Many algorithms have been developed in recent years; however, few prioritize speed over solution quality

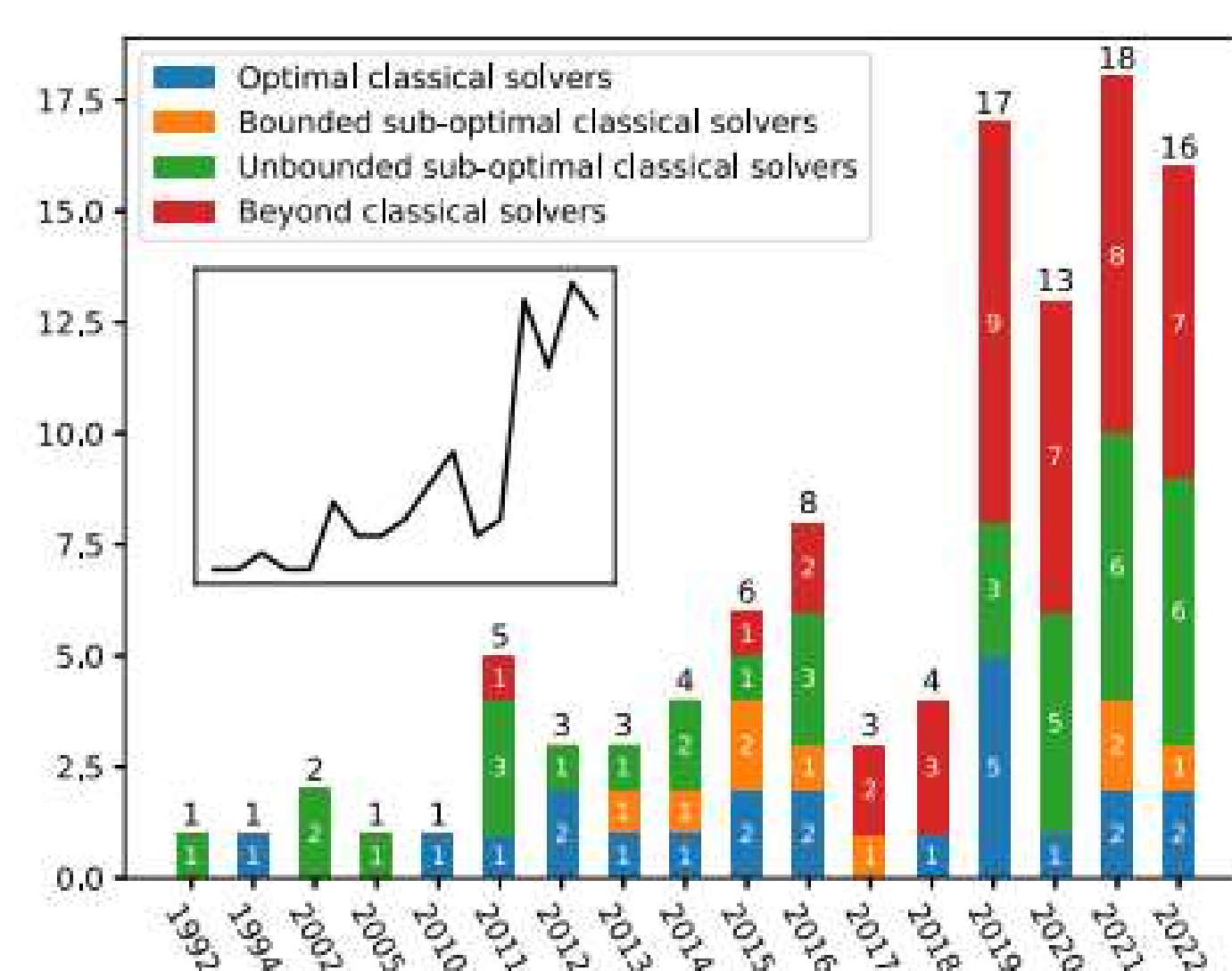


Figure 2. Development of MAPF algorithms over time, 1992-2022 (Gao et al., 2024)

Methods

- Final algorithm is a modified version of the CBS algorithm describe by Sharon et al. (2014) (see figure 3)

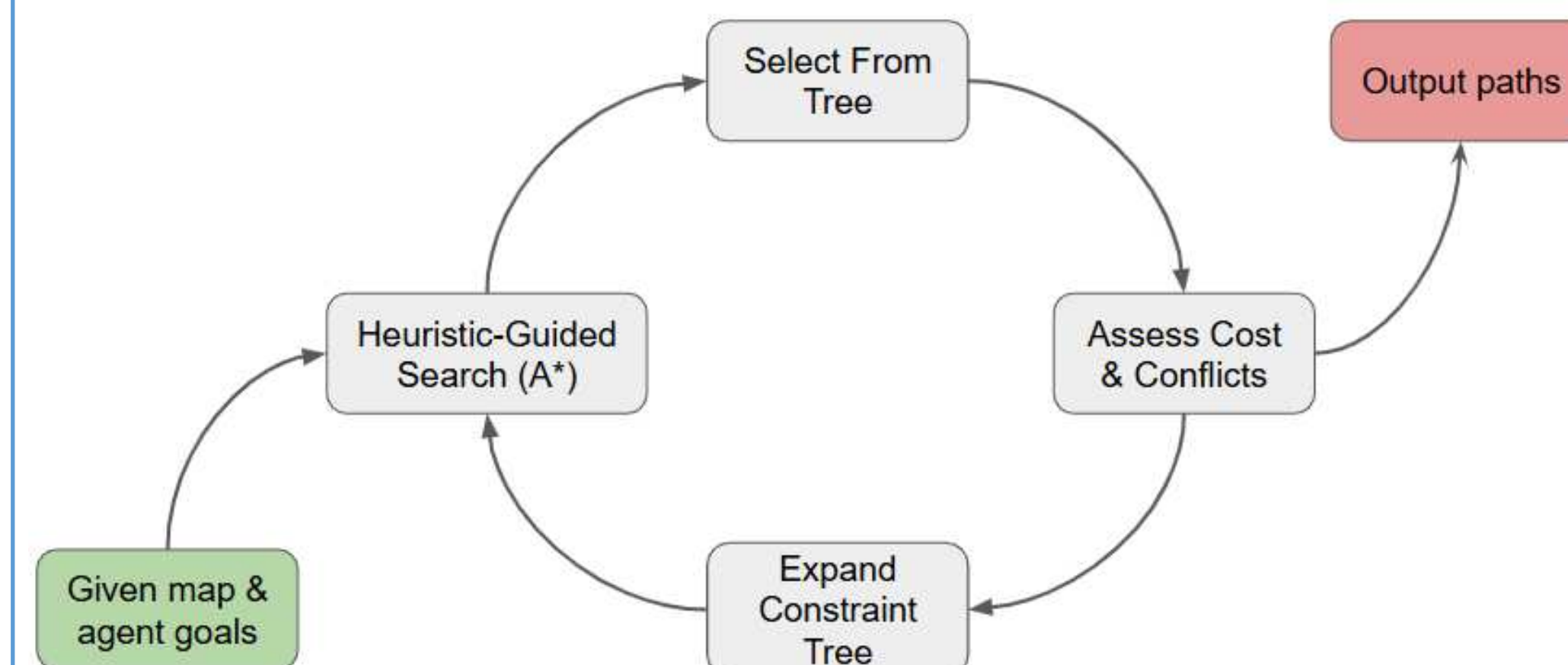


Figure 3. Flowchart of CBS, as described by Sharon et al. (2014)

- Standard CBS uses a 'constraint tree' system: each agent is planned separately, then improves iteratively by finding a conflict, splitting the problem into two subproblems, with each forcing one conflicting agent to replan, then repeating, creating a tree of possible solutions

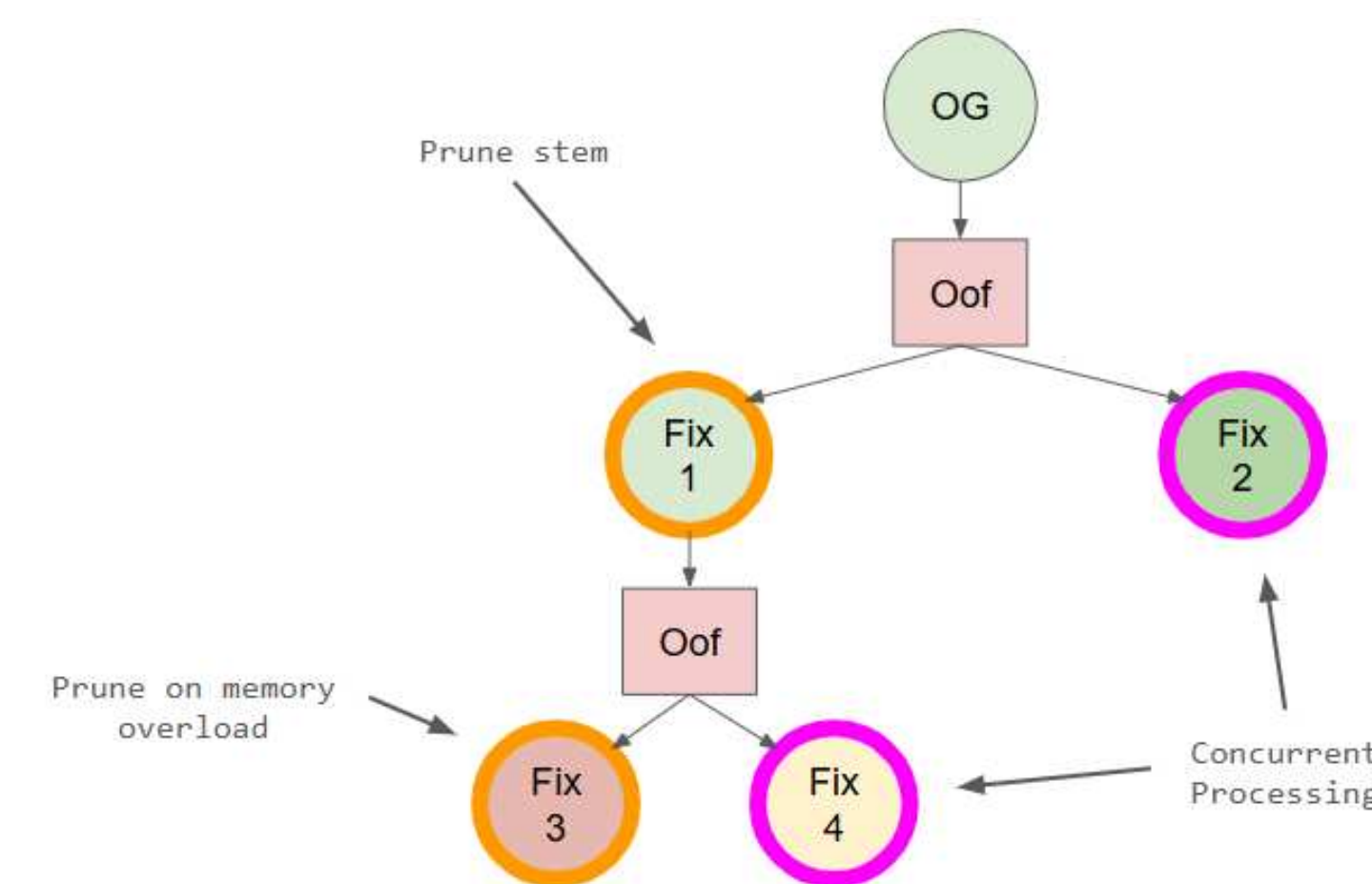


Figure 4. Diagram of final algorithm

- To attempt to improve performance, several branch nodes were processed concurrently to make use of the device's parallel processing capabilities
- Additionally, to prevent memory overload, branches were pruned instead of processed after the program reached a memory threshold. The threshold used for testing was the system's `max_concurrency()` scaled by a factor of 5, as this seemed to perform best
- This approach is likely feasible because MAPF problems often have many possible solutions, especially on relatively open maps such as that in Figure 1

Results

- Each test case was repeated 10,000 times on randomly generated maps with 20% of nodes replaced with obstacles
- Testing was performed on a Lenovo ThinkPad P15s, using an 11th gen Intel Core i7 processor and 2048 MiB heap memory
- The multithreaded algorithm was tested against a standard CBS implementation on five test scenarios:
 - A 16x16 map with 16 agents
 - A 32x32 map with 32 agents
 - A 64x64 map with 64 agents
 - A 128x128 map with 128 agents
 - An 8x128 map with 64 agents
- A failure threshold was set for each case; any time greater than the threshold or an algorithm terminating without a solution was counted as the threshold value in calculation of the average
- The mean times to complete each test case after 10,000 trials are displayed below:

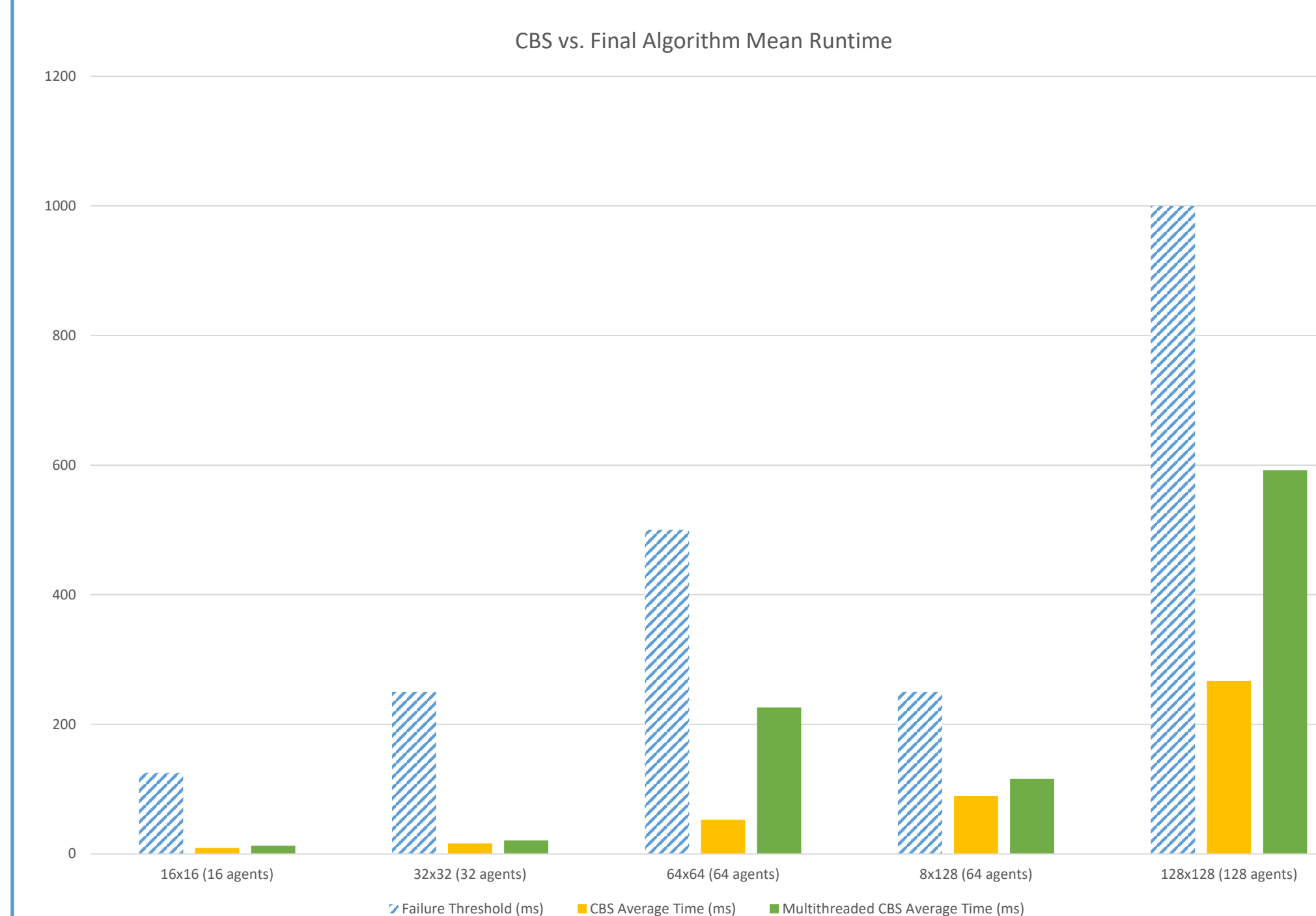


Figure 5. Failure penalty (blue), CBS mean runtime (yellow), and designed algorithm runtime (green) for each test case.

- CBS consistently averaged faster times than the designed algorithm across all five test cases
- The designed algorithm's mean runtime was often around half the failure penalty

Conclusions

- Since all test cases resulted in slower average run times with the multithreaded algorithm than the standard algorithm, the data suggest that aggressive memory pruning decreases the performance of multi-agent pathfinding algorithms
- The project did not yet succeed in designing a faster MAPF algorithm than others recently developed
- The designed algorithm, however, contained numerous parameters and a 'memory-threshold' function which were largely arbitrarily chosen
- It is possible, therefore, that further testing and/or problem-specific tweaking could yield performance better than standard CBS
- Preliminary research also produced numerous other techniques used to reduce the processing power requirement of MAPF algorithms, which it would most likely be possible to implement into the algorithm presented in this project, such as hierarchical pathfinding, agent grouping, and iterative improvement of solution quality

References

- Evtett, M., Hendler, J., Mahanti, A., and Nau, D. (1995) PRA*: Massively Parallel Heuristic Search. *Journal of Parallel and Distributed Computing*. 25(2). <https://doi.org/10.1006/jpdc.1995.1036>.
- Gao, J., Li, Y., Li, X., Yan, K., Lin, K., and Wu, X. (2024). A review of graph-based multi-agent pathfinding solvers: From classical to beyond classical. *Knowledge-Based Systems*. 283. <https://doi.org/10.1016/j.knosys.2023.111121>.
- Sharon, G., Stern, R., Felner, A., and Sturtevant, N. R. (December 2, 2014). Conflict-based search for optimal multi-agent pathfinding. *Artificial Intelligence*. 219(3). <http://dx.doi.org/10.1016/j.artint.2014.11.006>